# ESP-32 BASED RGB LED CONTROLLER

## Project Aim

To design and simulate an ESP32-based RGB LED controller with color selection and reset.

## Project Overview

This project uses an ESP32 to control a 16-LED NeoPixel RGB strip through two push buttons. One button selects LEDs while the other changes their colors independently. An I2C LCD displays the selected LED and color. The system supports color cycling, status monitoring, and complete reset functionality.

## Components Used

- ESP32
- NeoPixel Strip (16 LEDs)
- 2 Push Buttons
- 16×2 I2C LCD Display
- Jumper Wires

## Construction and Wiring Details

| Sl. No | Component | Pin/Terminal | ESP32 Pin |
|---|---|---|---|
| 1 | NeoPixel Strip | DIN | GPIO 5 |
| 2 | NeoPixel Strip | VCC | 5V |
| 3 | NeoPixel Strip | GND | GND |
| 4 | NEXT Button | Terminal 1 | GPIO 25 |
| 5 | NEXT Button | Terminal 2 | GND |

| Sl. No | Component | Pin/Terminal | ESP32 Pin |
|---|---|---|---|
| 6 | COLOR Button | Terminal 1 | GPIO 26 |
| 7 | COLOR Button | Terminal 2 | GND |
| 8 | I2C LCD | SDA | GPIO 21 |
| 9 | I2C LCD | SCL | GPIO 22 |
| 10 | I2C LCD | VCC | 5V |
| 11 | I2C LCD | GND | GND |

## Code Structure / Program Flow

```
┌─────────────────────┐
│   START   │
└──────────┬──────────┘
  │
  ▼
┌──────────────────────────┐
│ Initialize ESP32  │
│ NeoPixel Strip  │
│ LCD Display  │
└────────────┬─────────────┘
  │
  ▼
┌──────────────────────────┐
│ All LEDs OFF  │
```

```
│ Selected LED = 1    │
└─────────────┬───────────────┘
    │
    ▼
┌─────────────────────────────┐
│ Read Button States  │
└─────────────┬───────────────┘
    │
  ┌──────────┼────────────────┐
  │   │    │
  ▼   ▼    ▼
NEXT?  COLOR?  BOTH BUTTONS?
  │   │    │
 YES   YES    YES
  │   │    │
  ▼   ▼    ▼
Select  Change   Hold for
Next LED Color    2 Seconds?
  │   │    │
  ▼   ▼    ▼
Update  Update   RESET
LCD    LCD     SYSTEM
Serial  Serial
NeoPixel NeoPixel
  │   │    │
```

```
└────────────────┴────────────┬────────────┘
      │
      ▼
  ┌────────────────────────────────┐
  │ Repeat Loop │
  └────────────────┬───────────────┘
      │
      ▼
```

## Source Code

Diagram.jetson

```
{
 "version": 1,
 "author": "Aman abdulla",
 "editor": "wokwi",
 "parts": [
  {
   "type": "wokwi-breadboard",
   "id": "bb1",
   "top": -175.8,
   "left": 12.4,
   "attrs": { "color": "#eeefed" }
  },
  { "type": "board-esp32-devkit-c-v4", "id": "esp", "top": 57.6, "left": 273.64, "attrs": {} },
  {
   "type": "wokwi-pushbutton",
```

"id": "btn1",

"top": -96.4,

"left": 237.2,

"rotate": 270,

"attrs": { "color": "black", "xray": "1" }

},

{

"type": "wokwi-led-strip",

"id": "strip1",

"top": -238.3,

"left": 9.2,

"rotate": 270,

"attrs": { "pixels": "16" }

},

{

"type": "wokwi-lcd1602",

"id": "lcd1",

"top": 64,

"left": 504.8,

"attrs": { "pins": "i2c" }

},

{

"type": "wokwi-pushbutton",

"id": "btn3",

"top": -96.6,

"left": 329.4,

```
    "rotate": 90,
    "attrs": { "color": "green", "xray": "1" }
  }
],
"connections": [
  [ "esp:TX", "$serialMonitor:RX", "", [] ],
  [ "esp:RX", "$serialMonitor:TX", "", [] ],
  [ "bb1:19b.j", "esp:5V", "green", [ "v278.4" ] ],
  [ "bb1:18b.j", "esp:5", "red", [ "v67.2", "h182.4", "v124.8", "h28.8" ] ],
  [ "esp:GND.1", "bb1:17b.j", "black", [ "h-76.65" ] ],
  [ "esp:5V", "lcd1:VCC", "red", [ "h-28.65", "v28.8", "h249.6", "v-182.4" ] ],
  [ "esp:GND.2", "lcd1:GND", "black", [ "v0", "h124.8", "v9.6" ] ],
  [ "lcd1:SDA", "esp:21", "green", [ "h-19.2", "v9.8" ] ],
  [ "esp:22", "lcd1:SCL", "yellow", [ "h105.6", "v19.2" ] ],
  [ "strip1:VDD", "bb1:17b.i", "", [ "$bb" ] ],
  [ "strip1:DIN", "bb1:18b.i", "", [ "$bb" ] ],
  [ "strip1:VSS", "bb1:19b.i", "", [ "$bb" ] ],
  [ "esp:25", "bb1:24b.j", "green", [ "v0", "h-38.25" ] ],
  [ "esp:26", "bb1:34b.j", "green", [ "h-28.65", "v-134.4", "h105.6" ] ],
  [ "bb1:36t.b", "esp:GND.2", "green", [ "v0" ] ],
  [ "esp:GND.2", "bb1:26t.a", "black", [ "v-48", "h-76.8", "v-172.8" ] ],
  [ "btn1:1.l", "bb1:24b.h", "", [ "$bb" ] ],
  [ "btn1:2.l", "bb1:26b.h", "", [ "$bb" ] ],
  [ "btn1:1.r", "bb1:24t.c", "", [ "$bb" ] ],
  [ "btn1:2.r", "bb1:26t.c", "", [ "$bb" ] ],
  [ "btn3:1.l", "bb1:36t.c", "", [ "$bb" ] ],
```

```
    [ "btn3:2.l", "bb1:34t.c", "", [ "$bb" ] ],
    [ "btn3:1.r", "bb1:36b.h", "", [ "$bb" ] ],
    [ "btn3:2.r", "bb1:34b.h", "", [ "$bb" ] ]
  ],
  "dependencies": {}
}
```


Sketch.ino code

```
#include <Adafruit_NeoPixel.h>
#include <Wire.h>
#include <LiquidCrystal_I2C.h>

#define LED_PIN     5
#define NUM_LEDS    16

// Updated button pins
#define NEXT_BTN    25
#define COLOR_BTN   26

Adafruit_NeoPixel strip(NUM_LEDS, LED_PIN, NEO_GRB + NEO_KHZ800);
LiquidCrystal_I2C lcd(0x27, 16, 2);

int ledState[NUM_LEDS];
int selectedLED = 0;
```

```
bool lastNextState = HIGH;
bool lastColorState = HIGH;

unsigned long bothPressedStart = 0;
bool resetDone = false;

unsigned long blinkTimer = 0;
bool blinkState = false;

//------------------------------------------------
// Return RGB color corresponding to state
//------------------------------------------------
uint32_t getColor(int state) {
  switch (state) {
    case 0: return strip.Color(0, 0, 0);       // OFF
    case 1: return strip.Color(255, 0, 0);     // RED
    case 2: return strip.Color(0, 255, 0);     // GREEN
    case 3: return strip.Color(0, 0, 255);     // BLUE
    case 4: return strip.Color(255, 255, 0);   // YELLOW
    case 5: return strip.Color(255, 0, 255);   // PURPLE
    case 6: return strip.Color(0, 255, 255);   // CYAN
    case 7: return strip.Color(255, 255, 255); // WHITE
  }
  return strip.Color(0, 0, 0);
}
```

```
//------------------------------------------------
// Return color name
//------------------------------------------------
String colorName(int state) {
  switch (state) {
    case 0: return "OFF";
    case 1: return "RED";
    case 2: return "GREEN";
    case 3: return "BLUE";
    case 4: return "YELLOW";
    case 5: return "PURPLE";
    case 6: return "CYAN";
    case 7: return "WHITE";
  }
  return "OFF";
}

//------------------------------------------------
// Update LCD
//------------------------------------------------
void updateLCD() {
  lcd.clear();

  lcd.setCursor(0, 0);
  lcd.print("LED:");
  lcd.print(selectedLED + 1);
```

```
 lcd.setCursor(0, 1);
 lcd.print("Color:");
 lcd.print(colorName(ledState[selectedLED]));
}

//--------------------------------------------------
// Serial + LCD update
//--------------------------------------------------
void printStatus() {
 Serial.print("Selected LED: ");
 Serial.print(selectedLED + 1);
 Serial.print(" | Color: ");
 Serial.println(colorName(ledState[selectedLED]));

 updateLCD();
}

//--------------------------------------------------
// Update NeoPixels
//--------------------------------------------------
void updateLEDs() {

 for (int i = 0; i < NUM_LEDS; i++) {
  strip.setPixelColor(i, getColor(ledState[i]));
 }
```

```
  // Blink selected LED white
  if (blinkState) {
    strip.setPixelColor(selectedLED, strip.Color(255, 255, 255));
  }

  strip.show();
}

//--------------------------------------------------
// Reset System
//--------------------------------------------------
void resetSystem() {

  for (int i = 0; i < NUM_LEDS; i++) {
    ledState[i] = 0;
  }

  selectedLED = 0;

  Serial.println("SYSTEM RESET");

  lcd.clear();
  lcd.setCursor(0, 0);
  lcd.print("SYSTEM RESET");
```

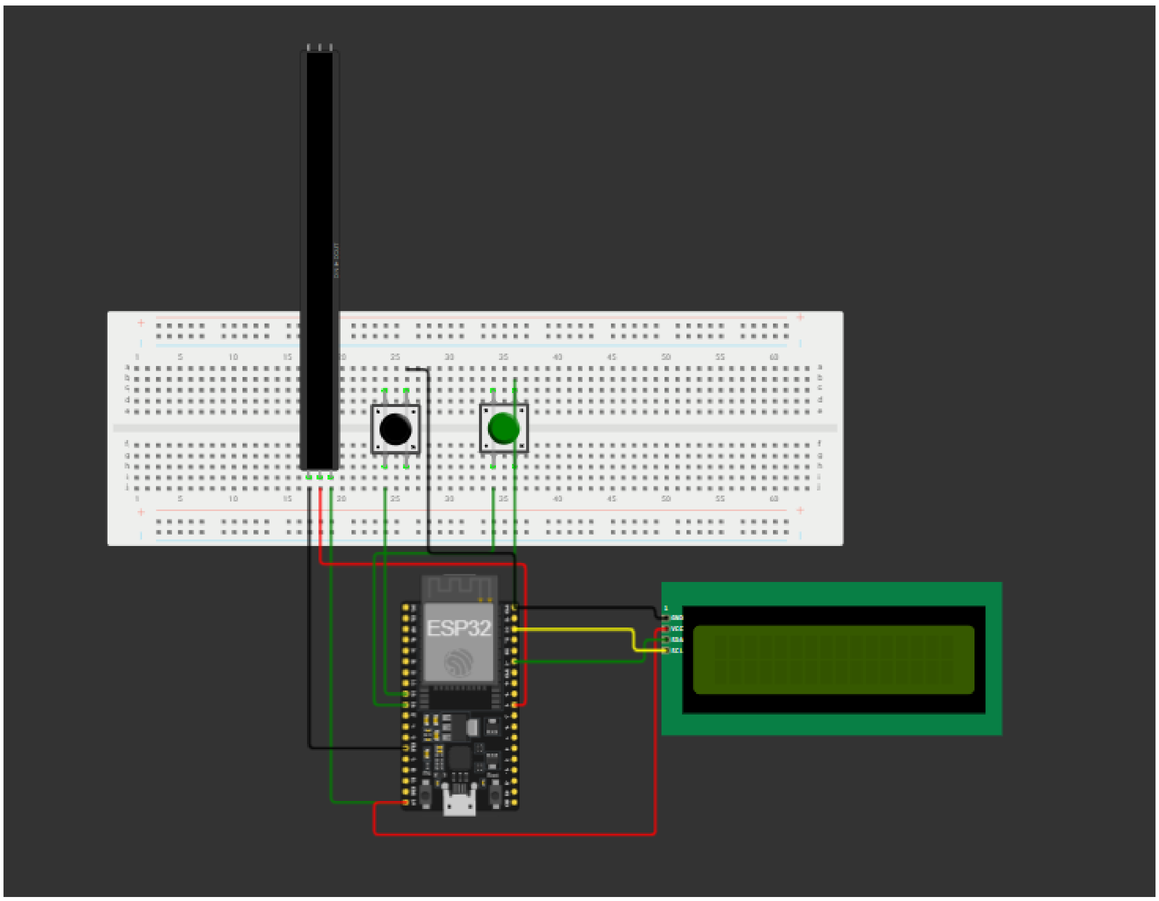
```
  strip.clear();
  strip.show();

  delay(1000);

  updateLCD();
}

//--------------------------------------------------
// Setup
//--------------------------------------------------
void setup() {

  Serial.begin(115200);

  pinMode(NEXT_BTN, INPUT_PULLUP);
  pinMode(COLOR_BTN, INPUT_PULLUP);

  strip.begin();
  strip.clear();
  strip.show();

  lcd.init();
  lcd.backlight();

  for (int i = 0; i < NUM_LEDS; i++) {
```

```
    ledState[i] = 0;
  }

  updateLCD();
  printStatus();
}

//--------------------------------------------------
// Main Loop
//--------------------------------------------------
void loop() {

  bool nextState = digitalRead(NEXT_BTN);
  bool colorState = digitalRead(COLOR_BTN);

  // Blink selected LED every 500ms
  if (millis() - blinkTimer >= 500) {
    blinkTimer = millis();
    blinkState = !blinkState;
    updateLEDs();
  }

  // Both buttons pressed for 2 seconds = RESET
  if (nextState == LOW && colorState == LOW) {

    if (bothPressedStart == 0) {
```

```
    bothPressedStart = millis();
  }

  if ((millis() - bothPressedStart >= 2000) && !resetDone) {
    resetSystem();
    resetDone = true;
  }
}
else {
  bothPressedStart = 0;
  resetDone = false;
}

// NEXT Button
if (lastNextState == HIGH &&
    nextState == LOW &&
    colorState == HIGH) {

  selectedLED++;

  if (selectedLED >= NUM_LEDS) {
    selectedLED = 0;
  }

  printStatus();
  updateLEDs();
```

```
    delay(200);
  }

  // COLOR Button
  if (lastColorState == HIGH &&
      colorState == LOW &&
      nextState == HIGH) {

    ledState[selectedLED]++;

    if (ledState[selectedLED] > 7) {
      ledState[selectedLED] = 0;
    }

    printStatus();
    updateLEDs();

    delay(200);
  }

  lastNextState = nextState;
  lastColorState = colorState;
}
```

## Project Photograph




Links

Google drive : https://drive.google.com/drive/folders/1mYpVfR_-E2dxaY-H-R-o99czcL6GqUu0?usp=sharing

Wokwi : https://wokwi.com/projects/466229218843430913

Git hub repo : https://github.com/AmanAZ5950/CIRCUITRON.git